

LLM PROCESSING PIPELINE

Technical Documentation

AI-Based News Summary Agent

Architecture, APIs, evaluation and operational design

Prepared by

Abisola Ajuwon

LLM Team Lead | Collaborative university software project

Document version	1.0
Document status	Technical documentation
Date	February 2026
AI-assisted development	Cursor
Repository	github.com/Abbysola/news-summary-agent

Scope note

This document focuses on the LLM processing and evaluation subsystem. The complete project documentation also covers the crawler, storage layer, frontend and detailed n8n workflow implementation.

Document Overview

This document consolidates the LLM pipeline draft, the standalone evaluation report and the complete system documentation into one implementation-oriented reference. It is intended for engineers, evaluators, project reviewers and recruiters who need to understand how the language-model subsystem is designed, executed and assessed.

Collaborative project and contribution

The news agent is a collaborative system. Abisola Ajuwon led the LLM team and coordinated the design, implementation, debugging and technical validation of the language-model components, including summarisation, clustering support, topic and keyword analysis, evaluation workflows and technical documentation. Cursor was used as the primary AI-assisted coding environment; architectural decisions, code review, integration, testing and final validation remained human-led.

Contents

1. Executive Summary
 2. Purpose, Scope and System Context
 3. End-to-End LLM Processing Lifecycle
 4. Semantic Representation and Clustering
 5. Summarisation Architecture
 6. Topic Interpretation, Keywords and Category Mapping
 7. Controlled Post-processing
 8. API Reference
 9. Evaluation Framework and Results
 10. Performance, Reliability and Operations
 11. Limitations and Future Development
- Appendix A:** Representative Prompt Templates
- Appendix B:** Glossary and References

1. Executive Summary

The LLM subsystem converts heterogeneous news articles into structured, multilingual and quality-scored intelligence. It combines statistical natural-language processing with locally served language models: SBERT embeddings create semantic representations; HDBSCAN and OpenSearch k-nearest-neighbour search support topic grouping; LDA and TF-IDF provide interpretable keyword signals; LLaMA generates summaries, labels and constrained stylistic rewrites; and MarianMT provides English-to-German translation.

The pipeline produces a hierarchy of artefacts: article summaries, cluster summaries, category-level digests and a global mega summary. n8n orchestrates the sequence, OpenSearch persists raw and derived artefacts, and FastAPI exposes the processing and evaluation functions as independently callable services.

Area	Current design	Why it matters
Semantic grouping	all-MiniLM-L6-v2 embeddings, HDBSCAN and incremental centroid matching	Finds coherent news topics without requiring a fixed number of clusters.
Summarisation	LLaMA 3.2 (3B) through Ollama, with hierarchical summary levels	Balances local detail with global coverage and predictable prompt constraints.
Interpretability	LDA and TF-IDF keywords plus LLM-generated labels	Combines statistical evidence with readable semantic descriptions.
Quality evaluation	BBC category benchmark and a three-judge LLM-as-a-Judge framework	Measures both taxonomic behaviour and higher-level summary quality.
Operational efficiency	Caching, parallel execution, incremental updates and local inference	Reduces repeated model calls and supports reproducible, privacy-preserving execution.

Evaluation highlights

Topic classification achieved 0.654 accuracy, 0.546 macro-F1 and 0.654 weighted-F1 on 1,490 BBC News articles. In the multi-judge summary evaluation, mega summaries outperformed cluster summaries on coherence (4.40 vs 3.95), consistency (4.37 vs 3.35), fluency (4.67 vs 4.11) and relevance (4.20 vs 3.76).

2. Purpose, Scope and System Context

2.1 Purpose and intended audience

The document explains the implementation decisions behind the LLM processing engine and its evaluation layer. It is written as a technical reference rather than a user manual. The primary audience is developers maintaining the service, reviewers evaluating architectural quality, and practitioners interested in reproducible local-LLM pipelines.

2.2 Scope boundaries

The LLM service operates within a larger news-agent architecture. The crawler, storage, orchestration and frontend layers are described only where they define an interface or operational dependency for the LLM pipeline.

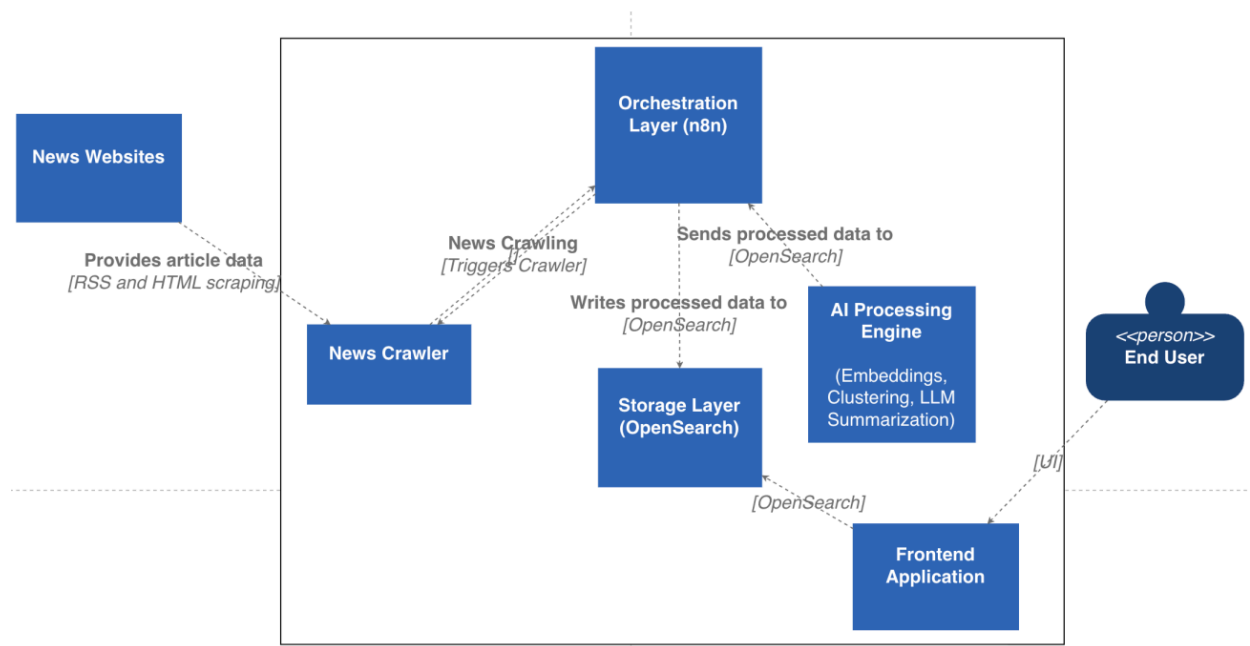


Figure 1. System context and component boundaries for the AI-based news agent.

2.3 Architectural responsibilities

Layer	Primary responsibility	Interaction with the LLM subsystem
Data acquisition	Fetch and validate articles from RSS feeds and web pages.	Supplies normalised article objects through the orchestration layer.
Orchestration (n8n)	Schedule workflows, sequence processing stages and handle integration.	Calls FastAPI endpoints and writes intermediate and final artefacts to OpenSearch.
Storage (OpenSearch)	Persist articles, embeddings, clusters, summaries, labels and evaluation results.	Provides cached inputs, k-NN similarity search and persistent derived artefacts.
AI processing engine	Translate, embed, cluster, summarise, label, classify, style and evaluate.	Core subject of this document.
Frontend (Streamlit)	Display clusters, summaries and translated or styled outputs.	Reads processed artefacts and allows user-facing selection of output forms.

2.4 Technology stack

Capability	Technology
API layer	Python and FastAPI
Local LLM serving	Ollama
Default summarisation and labelling model	LLaMA 3.2 (3B)
Semantic embeddings	Sentence-Transformers all-MiniLM-L6-v2 (384 dimensions)
Batch clustering	HDBSCAN
Incremental similarity matching	OpenSearch k-NN and cosine similarity
Statistical topic signals	LDA through Gensim; TF-IDF through scikit-learn
Text preprocessing	NLTK, regular expressions and NumPy
Translation	Helsinki-NLP/opus-mt-en-de through MarianMT
Evaluation judges	Qwen 2.5 7B Instruct, Mistral 7B Instruct and Gemma 2 9B

Capability	Technology
Workflow orchestration and persistence	n8n and OpenSearch
Containerisation	Docker

3. End-to-End LLM Processing Lifecycle

The LLM pipeline is organised as a sequence of independently callable stages. Each stage consumes structured input, generates a narrow output, and persists or forwards the result through n8n. This separation makes the workflow easier to test, replace and scale than a single monolithic service.

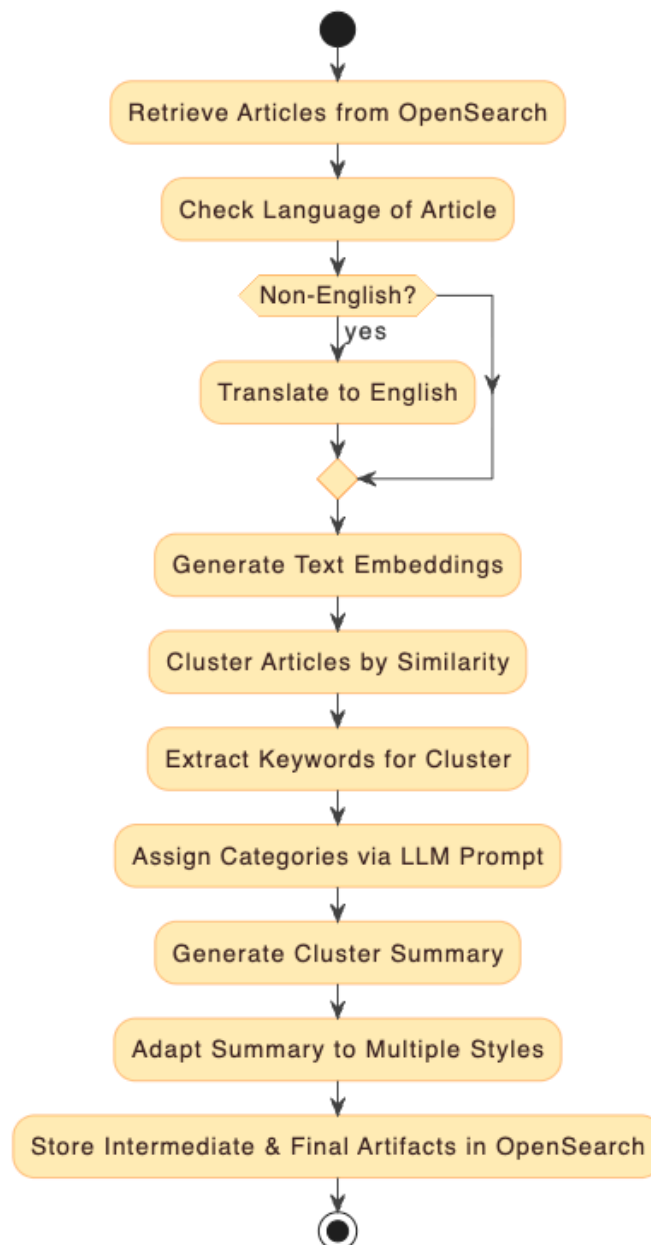


Figure 2. End-to-end LLM processing lifecycle.

3.1 Processing sequence

1. Retrieve new or updated article records from OpenSearch.
2. Inspect the language metadata and translate non-English articles to English.
3. Generate or retrieve a compact article summary to control input length.

4. Encode article text into normalised SBERT embeddings.
5. Assign articles to semantic clusters using HDBSCAN or incremental centroid matching.
6. Extract LDA and TF-IDF keywords as interpretable topic signals.
7. Generate a concise topic label and map the cluster to a fixed news category.
8. Generate cluster, category and mega summaries at increasing levels of abstraction.
9. Optionally apply style transformation or English-to-German translation.
10. Persist intermediate and final artefacts and, where configured, evaluate output quality.

3.2 Input normalisation

Articles arrive as structured JSON objects with identifiers, title, body, language, source and publication metadata. English is used as the internal processing language to keep embeddings, clustering and summarisation in one semantic space. German articles are translated before semantic processing, while original language metadata is retained.

Length-control strategy

all-MiniLM-L6-v2 truncates long inputs at approximately 256 WordPieces. The service therefore cleans and summarises articles before embedding. Article summaries are standardised to approximately 100 words, reducing token cost and producing more uniform clustering inputs.

3.3 Persisted artefacts

The storage design separates stable entities from derived artefacts. Articles remain the source records, while embeddings, cluster assignments, summaries, labels, keywords and evaluation outputs are stored independently. This allows the processing logic to evolve without retroactively changing raw content.

4. Semantic Representation and Clustering

4.1 SBERT embeddings

The pipeline uses the Sentence-Transformers model all-MiniLM-L6-v2 to map article text into a 384-dimensional dense vector space. Semantically related texts occupy nearby positions even when they share few surface-level keywords. The model was selected for its balance of semantic quality, throughput and memory efficiency compared with larger embedding models.

Embeddings are L2-normalised so that cosine similarity or dot product can be used directly. The same representation supports both batch clustering and incremental matching against stored centroids.

4.2 Batch clustering with HDBSCAN

HDBSCAN was selected because daily news streams contain an unknown and changing number of topics, uneven cluster sizes and noisy articles. Unlike K-Means, HDBSCAN does not require the number of clusters in advance and does not force every article into an assignment.

- Automatically infers the number of dense topic groups.
- Supports clusters with different shapes and densities.
- Marks weakly connected articles as noise rather than creating artificial groups.
- Produces cleaner inputs for downstream labelling and summarisation.

4.3 Noise and outlier handling

Noise articles are not discarded. They are routed to a General News pathway so that all valid articles remain represented while low-signal or mixed-topic items do not degrade the coherence of stronger clusters.

4.4 Incremental clustering

The first implementation re-embedded and re-clustered the complete article set for every update. The pipeline later adopted incremental clustering: each new article is compared with existing cluster centroids

using cosine similarity. A typical threshold of 0.75 determines whether the article joins an existing cluster or starts a new one. Centroids are then updated with a weighted running average.

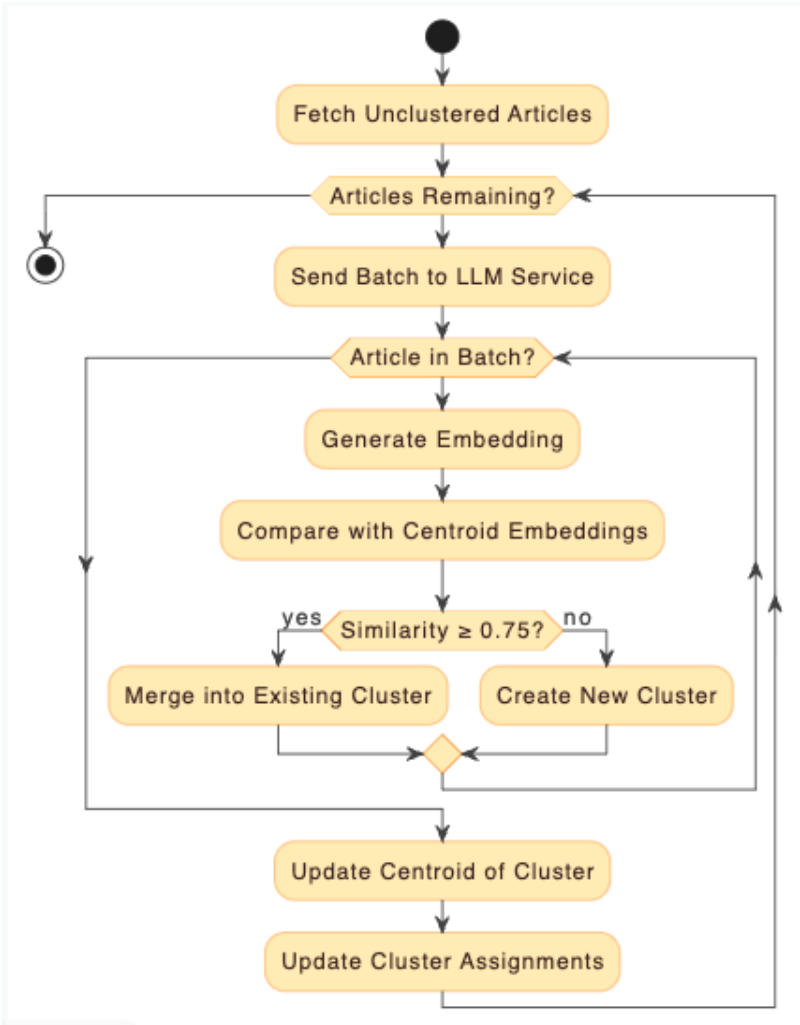


Figure 3. Incremental clustering with centroid matching and a typical 0.75 similarity threshold.

Operational effect

OpenSearch k-NN search moves similarity matching and centroid retrieval into the persistence layer. The backend remains stateless, only new articles require processing, and repeated full-dataset clustering is avoided.

4.5 Clustering endpoints

Endpoint	Purpose	Key behaviour
/cluster_create	Create cluster assignments from article inputs.	Translates if required, creates article summaries, embeds them with SBERT and applies HDBSCAN. Returns cluster IDs, article IDs and centroids.
/cluster_maintenance	Maintain persisted clusters.	Merges similar clusters, archives stale clusters and removes duplicates; may be scheduled or called manually.
/cluster_stats	Report cluster health and storage statistics.	Read-only access to active/archived counts, size distribution, category distribution, recent activity and storage metrics.

5. Summarisation Architecture

5.1 Baseline: BART

The first prototype used facebook/bart-large-cnn. BART provided a stable task-specific baseline but its 1,024-token input limit required overlapping chunks, repeated summarisation and a second compression pass. Although this validated the workflow and exposed token-limit bottlenecks, repeated hierarchical compression produced overly concise outputs and increased pipeline complexity.

5.2 Transition to LLaMA through Ollama

The current summariser uses LLaMA 3.2 (3B) through Ollama. The instruction-tuned model supports explicit constraints for target length, language, content coverage, redundancy avoidance, factuality and output format. The larger reported context window also reduces the need for repeated chunking at cluster and category levels.

Model-selection trade-off

The 3B model was selected as a compromise between instruction-following quality and local deployment cost. Larger models may improve absolute quality but introduce higher latency, memory requirements and operational complexity.

5.3 Hierarchical summary levels

Summary level	Input	Primary purpose
Article summary	One cleaned article.	Create a compact factual representation, normally around 100 words, for embedding and reuse.
Cluster summary	Article summaries or articles within one semantic cluster.	Remove repetition and preserve the shared facts, actors, locations and timeline.
Category digest	Cluster summaries belonging to one category.	Provide proportional coverage of related clusters within a domain.
Mega summary	Cluster summaries across all categories.	Produce a concise global briefing that prioritises important stories without allowing one topic to dominate.

5.4 Prompt controls

Summarisation prompts define the role, target word count, output language and required coverage. They also prohibit unsupported facts, repetition and extraneous formatting. The prompts instruct the model to preserve the main actors, locations and timeline and to return continuous prose rather than markdown.

5.5 Summarisation endpoints

Endpoint	Purpose	Design notes
/cluster_summarize	Generate a summary for one pre-clustered group.	Uses cached article summaries when available; generates missing article summaries on demand; returns the cluster summary only.
/mega_summarize	Aggregate cluster summaries into one high-level briefing.	Consumes pre-generated cluster summaries, avoiding article-level processing and reducing latency.

6. Topic Interpretation, Keywords and Category Mapping

6.1 Topic labelling

Numerical cluster IDs are not meaningful to users. The topic-labelling stage constructs a compact representation of each cluster from titles, leading sentences, article summaries, the cluster summary and optional keywords. A constrained LLaMA prompt then produces a title-case label of no more than four words, without punctuation or generic terms such as “news” or “update”.

The first prototype used LDA keywords as labels, but these were not sufficiently coherent as human-readable subcategories. LDA remains useful as supporting evidence, while the LLM supplies a concise semantic interpretation.

6.2 Statistical keyword extraction

Method	Process	Output
LDA	Treat sentences or cluster articles as documents, remove stopwords and identify latent topics. The default configuration extracts three topics and three keywords per topic.	Short multi-word topic signals, for example “pension reform government”.
TF-IDF	Preprocess the summary, score terms by TF-IDF and rank them by mean score.	A compact list of high-weight terms, for example “merz”, “pension”, “military”, “reform”.

6.3 Category mapping

Category mapping uses a constrained LLaMA prompt that selects exactly one predefined frontend category: Global Politics, Economics, Sports, Events or General News. LDA keywords can be injected into the prompt as statistical supporting signals. The model is instructed not to create new categories, and weak or noise clusters default to General News.

Hybrid interpretation strategy

The architecture separates structural grouping, statistical evidence and semantic naming: embeddings and HDBSCAN define cluster structure; LDA and TF-IDF expose keyword distributions; and LLaMA converts the combined evidence into readable labels and a fixed category assignment.

6.4 Interpretation endpoints

Endpoint	Purpose	Key behaviour
/topic_label	Create a concise label for a cluster or article summary.	Maximum four words, title case, no punctuation, no generic labels; input may be truncated for faster inference.
/keyword_extract	Extract LDA and TF-IDF keywords.	Accepts a summary and returns both statistical keyword sets.
/category_label	Assign a fixed frontend category.	Uses the cluster or article summary and optional LDA keywords; never invents categories; defaults weak/noise inputs to General News.

7. Controlled Post-processing

7.1 Style transformation

Style transformation is a post-processing stage that changes presentation without changing the underlying facts. The same factual summary can be adapted across three independent dimensions: editorial tone, writing style and output format.

Dimension	Examples	Constraint
Editorial tone	Neutral or institutional.	No advocacy, ideological framing or emotional loading.
Writing style	Journalistic, academic or executive.	Change expression and emphasis, not facts.
Output format	Paragraph, bullet points, TL;DR or short sections.	Preserve named entities, uncertainty and all factual content.

Prompts explicitly prohibit new facts, factual deletion and meaning changes. If no style parameters are supplied, the base summary is returned unchanged. This keeps presentation choices separate from core summarisation and reduces factual drift.

7.2 Translation

Translation is treated as a sequence-to-sequence transformation rather than an open-ended generation task. German articles are translated to English before embedding, clustering and summarisation. English cluster and mega summaries can later be translated to German for display, while the original English output remains available.

The display translation endpoints use Helsinki-NLP/opus-mt-en-de through MarianMT. Empty summaries produce an empty translated field, and all other payload attributes are preserved.

7.3 Post-processing endpoints

Endpoint	Purpose	Notes
/summary_style	Apply optional tone, style and format transformations.	Summarises once and performs controlled rewrites; returns the original summary when no options are supplied.
/translate_cluster_summary	Translate a cluster summary from English to German.	Adds summary_de without modifying other fields.
/translate_mega_summary	Translate a mega summary from English to German.	Uses the same MarianMT transformation and payload-preservation logic.

8. API Reference

The service exposes small, composable endpoints rather than a single all-in-one request. n8n combines these endpoints into scheduled and user-triggered workflows. The tables below summarise the main contracts described in the project documentation; the repository code remains the authoritative source for exact request schemas.

8.1 Processing and interpretation

Endpoint	Input concept	Output concept	Operational note
/cluster_create	Article records	Cluster IDs, article IDs, centroids	Batch clustering; designed for n8n integration.

Endpoint	Input concept	Output concept	Operational note
/cluster_maintenance	Persisted cluster state	Merge/archive/cleanup results	Safe maintenance operations; may create backups.
/cluster_stats	No modification request	Cluster and storage statistics	Read-only monitoring endpoint.
/cluster_summarize	Pre-clustered articles and optional cached summaries	Cluster summary	Avoids regeneration when cached summaries exist.
/mega_summarize	Pre-generated cluster summaries	One mega summary	Lightweight hierarchical aggregation.
/topic_label	Article or cluster summary	Concise topic label	Maximum four words.
/keyword_extract	Summary text	LDA and TF-IDF keyword lists	Statistical, non-generative analysis.
/category_label	Summary plus optional LDA keywords	One fixed category	Weak/noise inputs map to General News.

8.2 Post-processing and evaluation

Endpoint	Input concept	Output concept	Reliability behaviour
/summary_style	Base summary and optional style parameters	One or more controlled rewrites	No factual additions or removals; original returned if no options.
/translate_cluster_summary	Cluster summary object	Object with summary_de	Preserves all non-summary fields.
/translate_mega_summary	Mega summary object	Object with summary_de	Preserves all non-summary fields.
/evaluate_cluster	Cluster summary and source articles	Judge-level and aggregate scores	Partial aggregation allowed; fallback score of 3.0 if all judges fail.
/evaluate_mega	Mega summary and cluster summaries	Judge-level and aggregate scores	Same JSON validation, repair and fallback workflow.

9. Evaluation Framework and Results

Two complementary evaluation strategies are used. A supervised BBC News benchmark measures the behaviour of the topic-classification component against known labels. A multi-judge LLM-as-a-Judge workflow evaluates linguistic and semantic summary quality that is difficult to capture with surface-level metrics alone.

9.1 Topic-classification benchmark

Evaluation setup

The benchmark uses 1,490 English-language BBC News articles across business, entertainment, politics, sport and tech. Predictions were generated without exposing the true labels during inference, and results from 50 batch-level runs were aggregated. The benchmark taxonomy is used for external evaluation and is distinct from the production frontend categories.

Metrics

Accuracy provides overall correctness; precision measures the reliability of predicted category assignments; recall measures category coverage; and F1 balances precision and recall. Macro averages weigh each class equally, while weighted averages account for class frequency.

Class	Precision	Recall	F1	Support
Business	0.550	0.786	0.647	336
Entertainment	0.988	0.293	0.452	273
Politics	0.546	0.934	0.689	274
Sport	0.994	0.512	0.676	346
Tech	0.868	0.759	0.810	261
Accuracy			0.654	1,490
Macro average	0.658	0.547	0.546	1,490
Weighted average	0.788	0.654	0.654	1,490

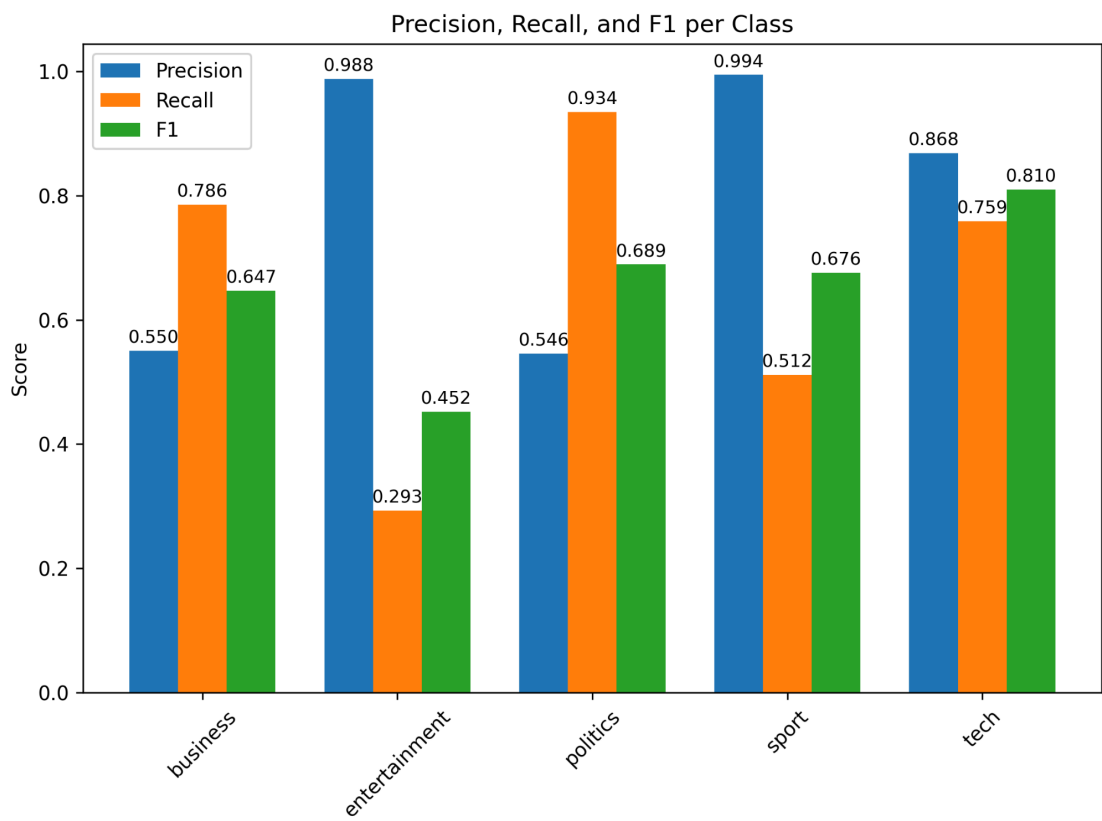


Figure 4. Precision, recall and F1 score by BBC News category.

Confusion patterns

The confusion matrix shows systematic rather than random errors. Business and politics act as semantic attractors: entertainment is frequently mapped to business or politics; sport is often mapped to politics; tech is commonly mapped to business; and business is sometimes mapped to politics. Unknown outputs represent malformed, empty or non-compliant responses rather than a true category.

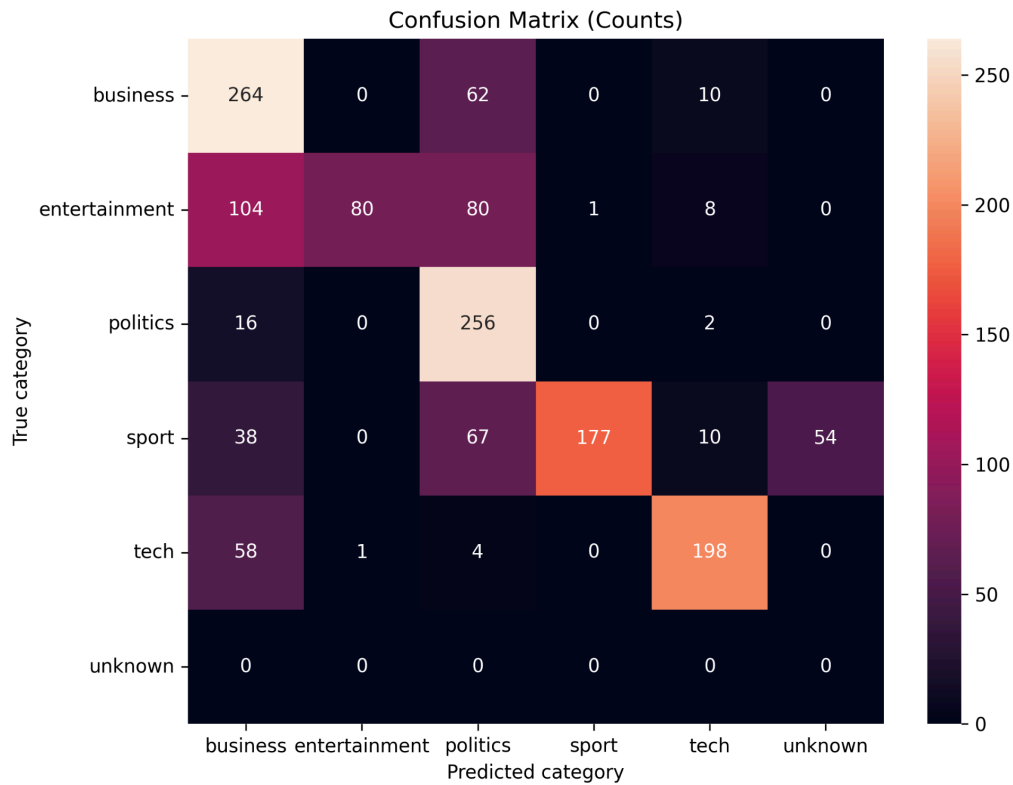


Figure 5. Topic-classification confusion matrix (counts).

Interpretation

Tech is the most balanced class (F1 = 0.810). Politics has very high recall (0.934) but lower precision (0.546), while entertainment and sport have extremely high precision but low recall. The model therefore favours semantic salience over strict taxonomic boundaries and assigns narrower categories conservatively.

Architectural implication

The findings support the hybrid topic architecture. LLM-based labelling alone is not sufficient for reliable fine-grained categorisation; embedding-based cluster structure and statistical topic signals reduce category absorption and make model decisions more interpretable.

9.2 Multi-judge LLM-as-a-Judge evaluation

Evaluation design

Ten live article batches were evaluated. Each batch contributed five category-level cluster summaries and one mega summary, producing approximately 60 evaluated summaries. Cluster summaries were compared with their source articles; mega summaries were compared with the underlying cluster summaries, matching the intended role of each output level.

The final judge panel consists of Qwen 2.5 7B Instruct, Mistral 7B Instruct and Gemma 2 9B. DeepSeek-R1-Distill-Qwen-32B was considered initially but was replaced because its reasoning-first behaviour did not consistently follow the strict output format required by the evaluation service.

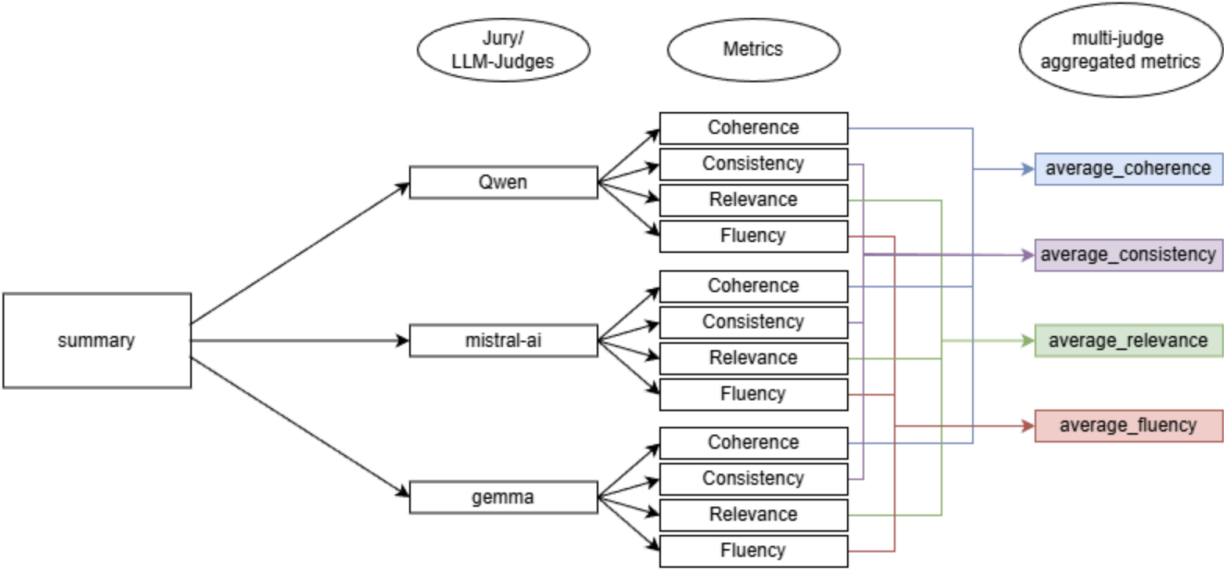


Figure 6. Multi-judge evaluation and metric aggregation architecture. The final implementation uses Gemma as the third judge.

Quality dimensions

Metric	Definition
Coherence	Logical flow, organisation and the collective quality of the summary.
Consistency	Factual alignment with the source and absence of contradictions or hallucinated claims.
Relevance	Preservation of important source information without redundancy or unimportant detail.
Fluency	Grammatical correctness, sentence quality and readability.

Metric	Definition
Tone (optional)	Alignment with the requested editorial tone.
Style (optional)	Alignment with the requested writing style and formatting constraints.

Scoring and failure handling

1. Each judge receives the source, summary, fixed rubric and strict JSON schema.
2. The service requests integer scores from 1 to 5 for each metric.
3. The response is parsed and validated. Invalid output triggers a repair prompt that asks the same model to convert its response into valid JSON.
4. Scores are averaged across all successful judges. One or two valid judges are sufficient for partial aggregation.
5. If every judge fails, the endpoint returns a neutral fallback score of 3.0 and records fallback status.
6. Raw failed outputs, error-stage identifiers and metadata are written to debug logs.

Summary-level results

Metric	Cluster summary	Mega summary	Difference
Coherence	3.95	4.40	+0.45
Consistency	3.35	4.37	+1.02
Fluency	4.11	4.67	+0.56
Relevance	3.76	4.20	+0.44

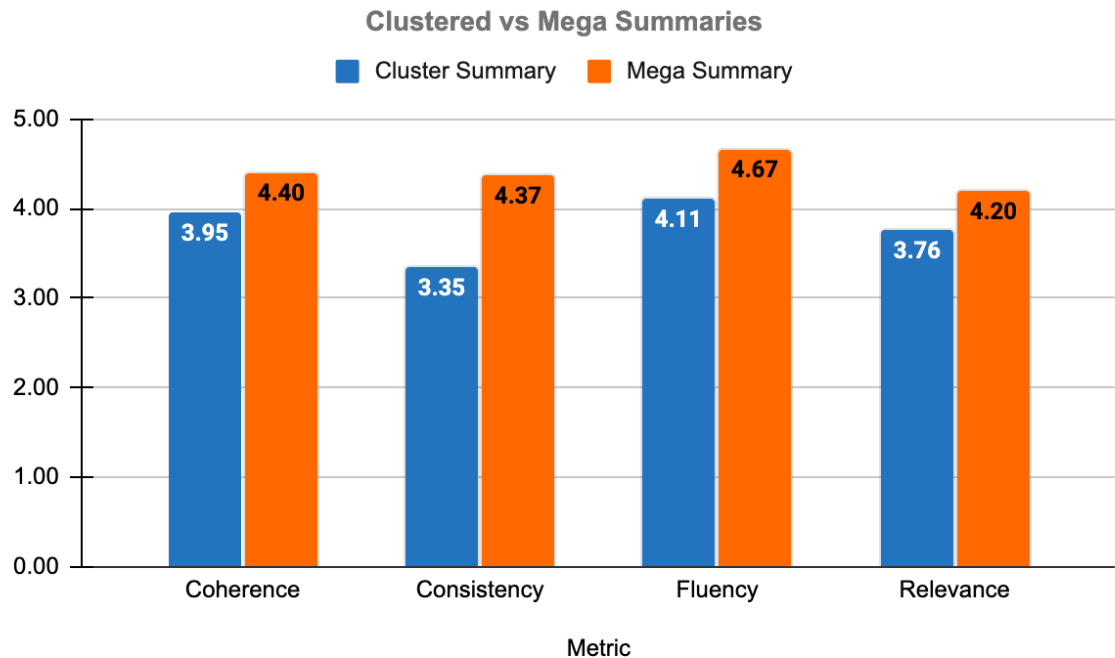


Figure 7. Mean LLM-as-a-Judge scores for cluster summaries and mega summaries.

Mega summaries score higher on every metric. The largest improvement is consistency (+1.02), followed by fluency (+0.56). The results suggest that hierarchical aggregation smooths local inconsistencies and produces a more stable global narrative rather than diluting relevant information.

Judge calibration and stability

The judges differ in absolute scoring tendencies. Mistral is the most generous, Qwen is more conservative, and the third judge shows greater variability. Despite these calibration differences, relative rankings between cluster and mega summaries remain stable. Multi-judge aggregation therefore reduces dependence on one model's scoring style.

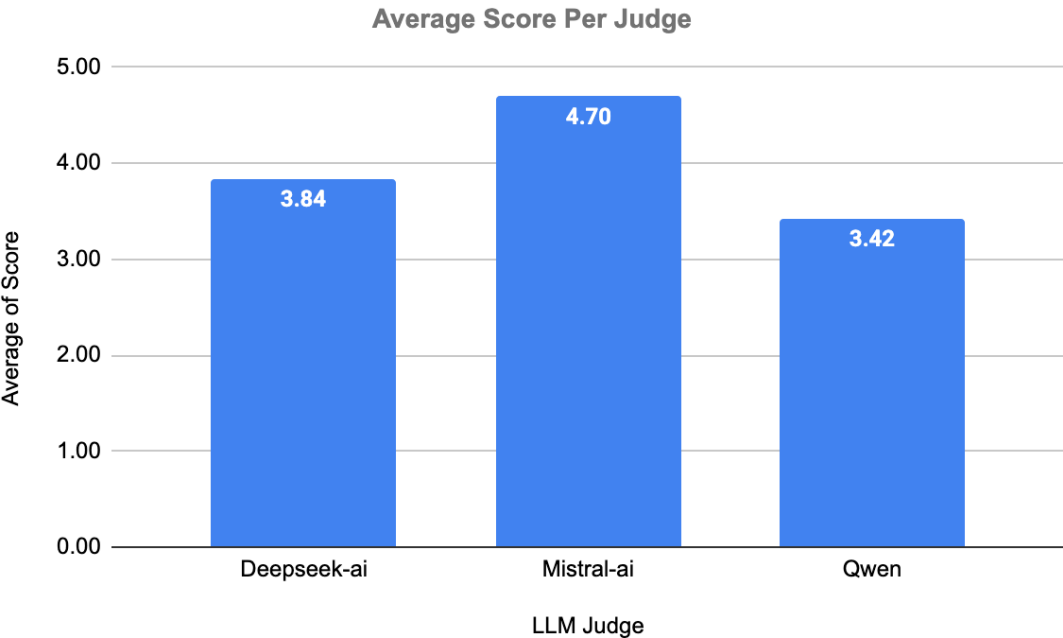


Figure 8. Average score by judge in the evaluation artefacts. The third-judge label in this legacy figure predates the final switch to Gemma.

Variance finding

Consistency (variance 1.55) and relevance (1.32) show greater judge disagreement than coherence (0.68) and fluency (0.67). Semantic alignment and information selection are therefore more subjective than surface-level language quality.

Live quality monitoring

The evaluation endpoints can be invoked for each new article batch. Cluster and mega summary scores are stored with the derived artefacts, enabling ongoing quality monitoring rather than evaluation only at the end of development.

10. Performance, Reliability and Operations

10.1 Runtime constraints

Early CPU-only testing exposed severe latency and stability problems. A 30-article batch could take approximately 3.5 hours, and a stalled Ollama generation could occupy a runtime slot indefinitely, block queued requests and trigger cascading API timeouts. Docker on Apple Silicon also prevented access to Metal GPU acceleration, forcing CPU-only inference.

10.2 Implemented optimisations

Optimisation	Implementation	Reported effect
Parallel translation	ThreadPoolExecutor with five workers.	Approximately fivefold reduction for batch translation.

Optimisation	Implementation	Reported effect
Translation cache	MD5 key over language, title and body; up to 1,000 cached entries.	Cache hits return immediately and reduce translation calls by an estimated 60-80%.
English early exit	Skip translation for English-language articles.	Removes unnecessary model calls for the dominant language.
Lazy MarianMT loading	Load the translation model on first use and retain it in memory.	Avoids unnecessary startup memory and repeated model initialisation.
Parallel summarisation	asyncio plus ThreadPoolExecutor with five workers.	Approximately fivefold batch speed-up.
Fixed article-summary length	Standardise article summaries to approximately 100 words.	Reduces generation cost, prevents downstream truncation and improves input consistency.
Summary caching	Retrieve existing article summaries from OpenSearch.	Eliminates repeated LLM calls in incremental and repeated runs.
OpenSearch k-NN matching	Use indexed nearest-neighbour search for centroids.	Provides scalable matching and persistent cluster state.

10.3 Performance summary

Deployment	Best case	Average	Per article	Success and stability
4 CPU, 15.6 GB RAM	19 min	21.9 min on clean runs; 35.2 min across all runs	~0.73 min clean; ~1.17 min overall	74.6% success (282/378); occasional issues under heavy concurrency.
12 CPU, 24 GB RAM, 16-core GPU available	1.84 min	2.03 min per 30 articles	4.1 sec	100% success; no timeouts in reported tests.

Deployment interpretation

The project documentation reports that the Docker deployment on macOS could not use the Apple GPU/Neural Engine. Native execution was substantially faster, while additional CPU resources improved both throughput and stability. Production deployment should therefore treat model-serving concurrency, hardware acceleration and request timeouts as first-class operational requirements.

10.4 Reliability controls

- Input validation before article processing and persistence.
- Idempotent storage using stable article identifiers in OpenSearch.
- Cached intermediate artefacts to reduce repeated model calls.
- Strict JSON schemas and repair prompts for evaluation outputs.
- Partial judge aggregation and neutral fallback status when evaluation models fail.
- Debug files containing raw outputs, failure stages and metadata.
- Cluster-maintenance operations for merging, archiving and duplicate cleanup.

11. Limitations and Future Development

Current limitation	Impact	Recommended direction
Fixed category taxonomy	Semantically overlapping articles may be absorbed by broad categories.	Use hierarchical classification, confidence scores or multi-label outputs where the product allows them.

Current limitation	Impact	Recommended direction
Prompt-format instability	Some LLMs may return malformed or non-compliant outputs.	Retain strict schemas, repair prompts and explicit unknown/fallback states; add schema-constrained decoding where supported.
Static similarity threshold	A single 0.75 threshold may not fit all categories or changing news volumes.	Calibrate thresholds on historical data and monitor cluster cohesion and drift.
CPU-bound local inference	High latency and timeout risk under concurrent load.	Use hardware acceleration, controlled request queues, model-serving timeouts and capacity-based concurrency.
Subjective judge calibration	Different judge models have different leniency and variance.	Track judge-level scores, calibrate against human ratings and report uncertainty alongside averages.
Category-dependent summary quality	Heterogeneous categories such as General News show lower stability.	Adopt category-aware prompts and use cluster quality signals to decide aggregation strategy.
Limited interpretability in the interface	Users cannot directly inspect embedding geometry or prompt variants.	Add semantic-space visualisation, cluster diagnostics and side-by-side prompt/model comparisons.
Evolving topic streams	Centroids may become stale as stories develop.	Introduce topic-drift detection, time-aware centroid updates and dynamic cluster lifecycle rules.

The prototype is intentionally modular and research-oriented. Its strongest foundation is the separation of semantic structure, language generation, orchestration and evaluation, which allows individual components to be replaced or extended without redesigning the complete system.

Appendix A. Representative Prompt Templates

The following templates are condensed from the implementation documentation. Placeholders are shown in braces. Exact repository prompts remain the authoritative implementation reference.

A.1 Article summarisation

You are a news summarisation assistant.

Write a single coherent summary in {language} of about {target_words} words.
Focus on the main facts, actors, locations and timeline.
Preserve key information and avoid repetition.
Do not invent information that is not in the text.
Write in continuous prose, with no bullet points or markdown.

ARTICLE:
{text}

SUMMARY:

A.2 Cluster summarisation

You are a news summarisation assistant.
All articles describe the same news topic.

Write a single coherent summary in {language} of about {target_words} words.
Focus on the main facts, actors, locations and timeline.
Avoid repetition and do not invent information that is not in the text.
Write in continuous prose, with no bullet points or markdown.

TEXT:
{text}

SUMMARY:

A.3 Topic label

You are a news editor.
Generate a short topic label for the following news summary.
The label must be at most {max_words} words.
Use title case.
Do not use punctuation.
Do not mention “news”, “report” or “update”.

SUMMARY:
{truncated_summary}

TOPIC LABEL:

A.4 Fixed category assignment

You are a news editor categorising a news topic.
Choose the SINGLE best category from the list below.
Do NOT create new categories.
Return ONLY the category name, exactly as written.

CATEGORIES:
- Global Politics
- Economics
- Sports
- Events
- General News

CLUSTER SUMMARY:
{summary}

LDA KEYWORDS:
{lda_keywords}

CATEGORY:

A.5 LLM-as-a-Judge response contract

```
{
  "coherence": 1|2|3|4|5,
  "consistency": 1|2|3|4|5,
  "relevance": 1|2|3|4|5,
  "fluency": 1|2|3|4|5
}
```

The judge prompt instructs each model to compare the summary with the source, apply the full 1-5 scale and return only valid JSON. Optional tone and style metrics are evaluated in separate calls so that they do not affect the four standard quality scores.

Appendix B. Glossary and References

B.1 Glossary

Term	Meaning in this project
Article summary	Compact summary of one source article, typically around 100 words.
Cluster	A group of semantically related news articles.
Cluster summary	Summary that consolidates the shared information in one cluster.
Category digest	Aggregate summary of clusters mapped to one frontend category.
Mega summary	Global briefing aggregated from cluster summaries across categories.
Centroid	Representative vector used to compare new articles with an existing cluster.
Noise article	Article not strongly associated with a dense HDBSCAN cluster.
LLM-as-a-Judge	Use of language models to score generated outputs against a rubric.

Term	Meaning in this project
Partial aggregation	Average computed from the subset of judge models that returned valid results.
Fallback	Neutral score and status returned when no judge provides a valid result.

B.2 Selected references

BBC News classification dataset: <https://www.kaggle.com/competitions/learn-ai-bbc/data>

Fabbri et al. (2020), SummEval: Re-evaluating Summarization Evaluation: <https://arxiv.org/abs/2007.12626>

Liu et al. (2023), G-EVAL: NLG Evaluation using GPT-4 with Better Human Alignment: <https://arxiv.org/abs/2303.16634>

Verga et al. (2024), Replacing Judges with Juries: <https://arxiv.org/abs/2404.18796>

Gu et al. (2024), A Survey on LLM-as-a-Judge: <https://arxiv.org/abs/2411.15594>

Evaluating Small Language Models for News Summarization: <https://arxiv.org/abs/2502.00641>

Unstructured Evidence Attribution for Long Context Query-Focused Summarization: <https://arxiv.org/abs/2502.14409>

Project repository: <https://github.com/Abbysola/news-summary-agent>